

PRM3

Théorie : L'outil SeqInt

Séquences d'entiers

Outil SeqInt

Itérateur

Exemples d'utilisation

Séquence

Structure qui sera utilisée pour exemplifier la méthodologie des hypothèses inductives (voir plus loin).

Collection ordonnée d'éléments de même type (ici entiers)

Exemple : [12, 4, -20, 8]

$[e_1, e_2, \dots, e_n]$ où les

$$e_i \in \mathbb{Z}$$

Séquence

L'ordre des éléments est important

$$[5, -6, 7, 16, -3] \neq [-6, 7, 5, 16, -3]$$

La taille d'une séquence est le nombre d'éléments de la séquence. Par exemple, la taille des 2 séquences ci-dessus est de 5.

Remarque : une séquence d'un élément est différente de l'élément lui-même. La séquence $[7]$ est différente du nombre 7.

Séquence vide : séquence ne contenant aucun élément :

$[\]$

Outil SeqInt

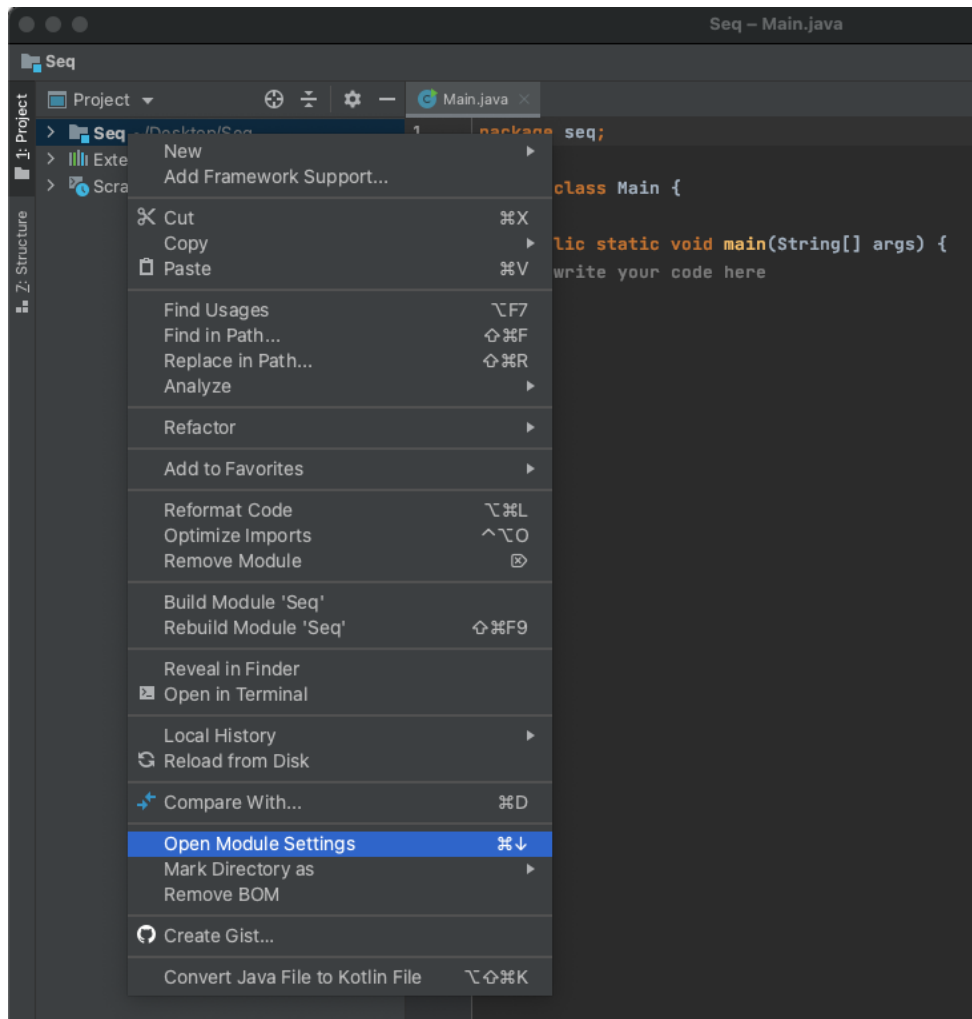
Outil créé pour l'utilisation de séquence d'entiers. Il est défini dans une librairie à inclure dans votre projet.

Pour incorporer cette librairie, vous devez copier le fichier `SeqInt.jar` dans le dossier de votre projet

Ensuite, vous indiquez à IntelliJ que vous utilisez cette librairie (voir captures d'écrans suivantes)

Outil SeqInt

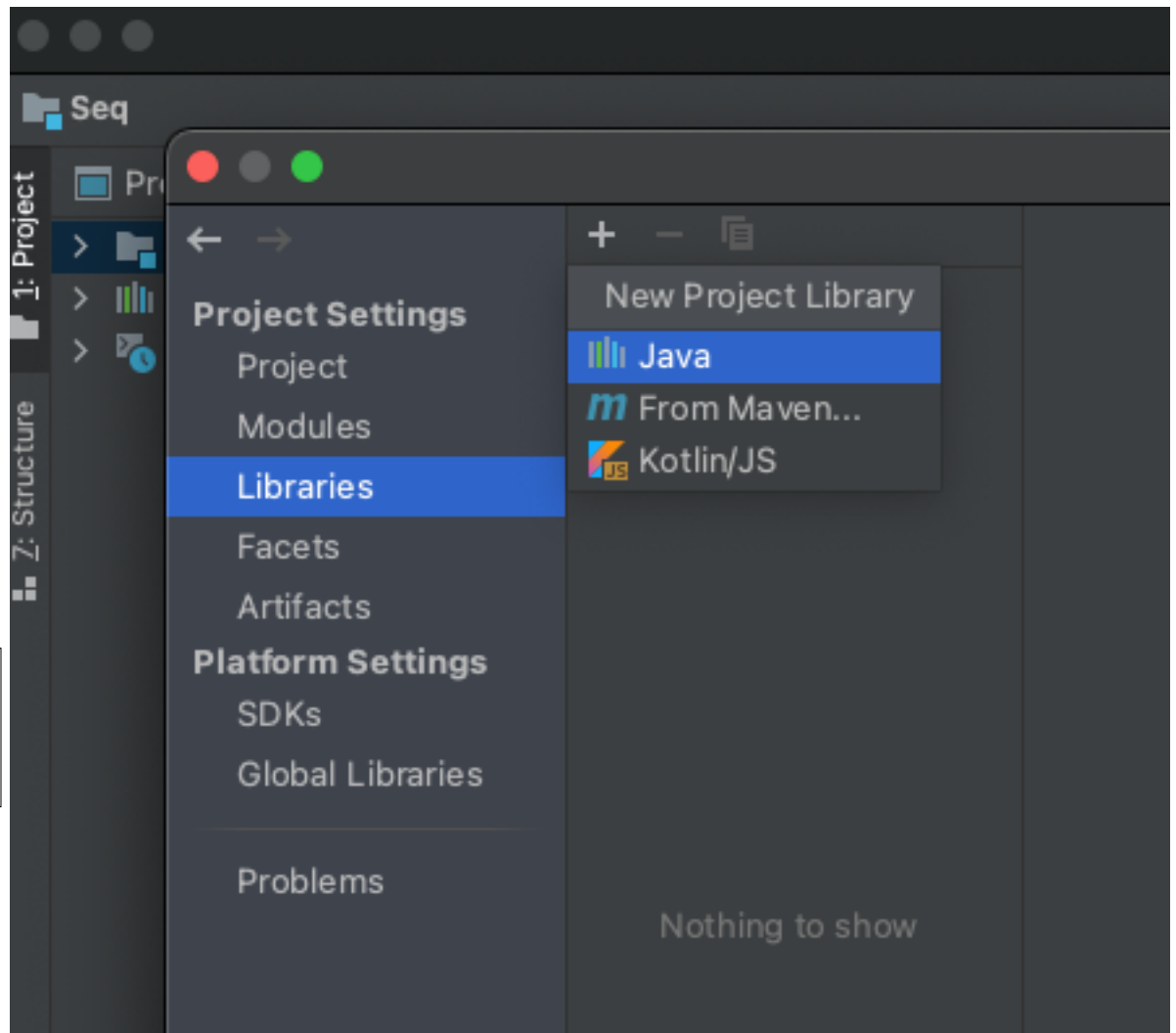
Cliquez-droit sur le projet



Cliquez sur
Open Module
Settings

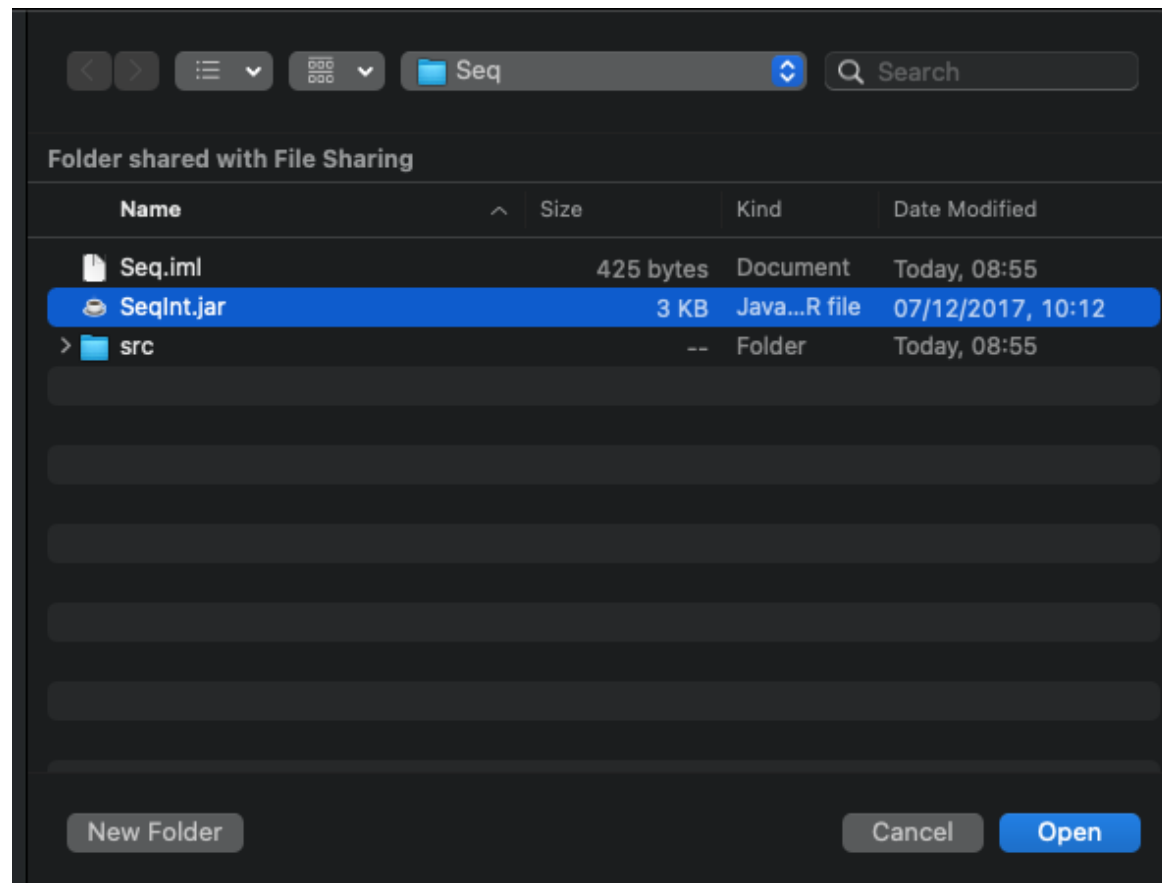
Outil SeqInt

Cliquez sur Librairies
puis sur « + » / « Java »



Outil SeqInt

Sélectionnez
le fichier



Cliquez
sur Ouvrir

Outil SeqInt - Utilisation

Dans vos programmes, vous pouvez créer des séquences définies par vous-même ou aléatoires (déterminées par l'outil).

```
SeqInt s1 = new SeqInt(5, 2, 1, 1, 1);
```

Création d'une séquence précise

```
SeqInt s2 = new SeqInt();
```

Création d'une séquence vide

```
SeqInt s3 = SeqInt.randomInstance();
```

Création d'une séquence aléatoire

Itérateur

Quelles sont les opérations permises sur une séquence ?

On peut **parcourir** la séquence : c'est à dire lire les éléments les uns après les autres

Pour cela, on utilise un ***Itérateur***.

On associe l'itérateur à la séquence

L'itérateur est une sorte de *fenêtre* sur la séquence. Il permet de lire l'élément courant de celle-ci (et d'avancer la fenêtre)

Itérateur

On déclare une variable de type SeqIntIterator:

```
SeqIntIterator it = s.iterator();
```

l'itérateur it est associé à la séquence s

Deux opérations sont possibles sur it :

Vérifier s'il y a un encore élément à parcourir dans la séquence :

```
it.hasNext() (retourne un booléen).
```

Lire l'élément sur lequel est positionné l'itérateur. Cette opération a pour conséquence de faire avancer l'itérateur au-dessus de l'élément suivant de la séquence :

```
int x = it.next();
```

Programme utilisant SeqInt

```

import seqint.SeqInt;
import seqint.SeqIntIterator;

public class Sequences_exemples {

    public static void main(String[] args) {
        SeqInt s = new SeqInt(5, 2, 1, 7, 7, 1, 2);
        SeqIntIterator it = s.iterator();

        // Boucle sur les éléments de la séquence
        while (it.hasNext()) {           // Tant qu'il y a un élément
            int x = it.next();           // Obtenir cet élément
            // Faire quelque chose avec l'élément
        }
        // Suite éventuelle
    }
}

```

Exemple 1

Écrire un sous-programme qui affiche les éléments d'une séquence

```
public static void affiche(SeqInt s) {
    SeqIntIterator it = s.iterator();
    System.out.print("[");
    while (it.hasNext()) {
        System.out.print(it.next() + " ");
    }
    System.out.println("]");
}
```

On reçoit la séquence
en paramètre

Itérateur sur la
séquence

Tant qu'il y a des
éléments

Obtenir et utiliser cet élément

Remarque : Ce programme n'est montré que pour l'exemple car vous pourrez plus simplement afficher une séquence en utilisant l'instruction `System.out.println(s);`

Exemple 1 - Version alternative

```
public static void affiche(SeqInt s) {
    SeqIntIterator it = s.iterator();
    System.out.print("[");
    if (it.hasNext()) {
        System.out.print(it.next());
        while (it.hasNext()) {
            System.out.print(", " + it.next());
        }
    }
    System.out.println("]");
}
```

Si il y a un premier
élément

On l'affiche

Pour les autres éléments

L'affichage est précédé
d'une virgule

Remarque : Ce programme n'est montré que pour l'exemple car vous pourrez plus simplement afficher une séquence en utilisant l'instruction `System.out.println(s);`

Exemple 2

Écrire un sous-programme qui retourne la longueur d'une séquence

```

public static int longueur(SeqInt s) {
    SeqIntIterator it = s.iterator();
    int lg = 0;          // initialement, la longueur de la séquence est zéro
    while (it.hasNext()) {
        it.next();       // Pour chaque nouvel élément
        ++lg;            // On incrémente la longueur
    }
    return lg;           // On retourne la longueur
}

```